MISSION FORGE

DICTIONNAIRE DES DONNÉES COMMUNES

Version 1.0 — Document de référence pour les cinq membres

1. OBJECTIF DU DOCUMENT

Ce document définit les noms et les types de données que tous les membres de l'équipe doivent utiliser dans leurs modules.

Chaque membre développe une partie différente du jeu. Cependant, les différents modules doivent pouvoir communiquer entre eux.

Pour éviter les problèmes d'intégration, nous devons utiliser les mêmes noms de propriétés et les mêmes types de données.

Règle fondamentale : lorsqu'un nom est défini dans ce document, tous les membres doivent utiliser exactement le même nom et le même type.

---

2. CONVENTIONS GÉNÉRALES

Élément	Convention à respecter	Exemple
Nom d'une propriété	camelCase	`totalMass`
Nom d'une interface	PascalCase	`Spacecraft`
Nom d'un identifiant	`id`	`id: string`
Texte	`string`	`"Explorer"`
Nombre	`number`	`850`
Vrai ou faux	`boolean`	`true`
Liste	`Type[]`	`componentIds: string[]`

Les noms doivent être écrits exactement comme dans les tableaux suivants.

---

3. LES STRUCTURES COMMUNES

A. MISSION

Représente une mission spatiale proposée au joueur.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant de la mission
`name`	`string`	Nom de la mission
`description`	`string`	Description de la mission
`objective`	`string`	Objectif à atteindre
`difficulty`	`MissionDifficulty`	Niveau de difficulté
`destination`	`string`	Destination spatiale
`durationDays`	`number`	Durée prévue en jours
`budgetLimit`	`number`	Budget maximal

Nom exact de la propriété	Type	Signification
`maxSpacecraftMass`	`number`	Masse maximale autorisée

B. SPACECRAFT

Représente le vaisseau spatial construit par le joueur.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant du vaisseau
`name`	`string`	Nom du vaisseau
`missionId`	`string`	Identifiant de sa mission
`componentIds`	`string[]`	Liste des composants installés
`totalMass`	`number`	Masse totale
`totalCost`	`number`	Coût total
`availableEnergy`	`number`	Énergie disponible
`fuel`	`number`	Quantité de carburant
`communicationRangeKm`	`number`	Portée de communication en kilomètres

C. COMPONENT

Représente un composant pouvant être installé dans le vaisseau.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant du composant
`name`	`string`	Nom du composant
`category`	`ComponentCategory`	Catégorie du composant
`description`	`string`	Description
`mass`	`number`	Masse
`cost`	`number`	Prix
`powerConsumption`	`number`	Énergie consommée
`powerProduction`	`number`	Énergie produite
`fuelCapacity`	`number`	Capacité de carburant
`reliability`	`number`	Fiabilité

D. MISSION EVENT

Représente un événement imprévu pendant une mission.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant de l'événement
`name`	`string`	Nom de l'événement

Nom exact de la propriété	Type	Signification
`description`	`string`	Description
`probability`	`number`	Probabilité d'apparition
`impact`	`ImpactLevel`	Niveau de gravité
`affectedResources`	`ResourceType[]`	Ressources affectées

E. SIMULATION RESULT

Représente le résultat d'une simulation.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant du résultat
`missionId`	`string`	Mission concernée
`spacecraftId`	`string`	Vaisseau concerné
`success`	`boolean`	Mission réussie ou non
`score`	`number`	Score obtenu
`durationDays`	`number`	Durée réelle
`resourcesAtEnd`	`ResourceState`	État final des ressources
`eventsEncountered`	`MissionEvent[]`	Événements rencontrés
`summary`	`string`	Résumé de la simulation

F. MISSION AUTOPSY

Représente l'analyse de la mission après la simulation.

Nom exact de la propriété	Type	Signification
`simulationResultId`	`string`	Identifiant du résultat analysé
`mainCauses`	`string[]`	Causes principales du résultat
`goodDecisions`	`string[]`	Bonnes décisions prises
`badDecisions`	`string[]`	Mauvaises décisions prises
`lessonsLearned`	`string[]`	Leçons à retenir

G. GAME PLAYER

Représente un participant à une partie.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant du joueur
`displayName`	`string`	Nom affiché
`role`	`PlayerRole`	Rôle du joueur
`isReady`	`boolean`	Joueur prêt ou non

H. GAME ROOM

Représente une salle de jeu multijoueur.

Nom exact de la propriété	Type	Signification
`id`	`string`	Identifiant de la salle
`name`	`string`	Nom de la salle
`missionId`	`string`	Mission sélectionnée
`hostPlayerId`	`string`	Identifiant de l'hôte
`players`	`GamePlayer[]`	Liste des joueurs
`status`	`RoomStatus`	État de la salle
`createdAt`	`string`	Date de création

---

4. LES TYPES À UTILISER POUR CERTAINES PROPRIÉTÉS

Certaines propriétés ne peuvent prendre que des valeurs précises.

Nom du type	Valeurs autorisées
`MissionDifficulty`	`"easy"`, `"medium"`, `"hard"`
`ComponentCategory`	`"structure"`, `"propulsion"`, `"power"`, `"communication"`, `"scientific_instrument"`, `"thermal_control"`, `"other"`
`ImpactLevel`	`"low"`, `"medium"`, `"high"`, `"critical"`
`PlayerRole`	`"commander"`, `"engineer"`, `"scientist"`, `"communicator"`
`RoomStatus`	`"waiting"`, `"designing"`, `"simulating"`, `"finished"`
`ResourceType`	`"mass"`, `"energy"`, `"fuel"`, `"budget"`, `"data"`, `"communication"`, `"crew"`

Exemple : si un membre veut définir la difficulté d'une mission, il doit utiliser `"easy"`, `"medium"` ou `"hard"`, et non inventer `"facile"` ou `"difficile"`.

---

5. EXEMPLE CONCRET : COMMENT LES MEMBRES DOIVENT COLLABORER

Imaginons que le membre 2 construise un vaisseau.

Il utilise obligatoirement les propriétés définies dans `Spacecraft` :

```
const spacecraft = {
  id: "ship-001",
  name: "Explorer",
  missionId: "mission-001",
  componentIds: ["engine-01"],
  totalMass: 850,
  totalCost: 120000,
  availableEnergy: 500,
  fuel: 300,
  communicationRangeKm: 100000
};
```

Le membre 3 récupère ce même objet pour effectuer la simulation.

Il doit donc lire `totalMass`, et non `massTotal`, `masseTotale` ou un autre nom.

Le membre 1 peut ensuite afficher la valeur `totalMass` dans l'interface.

Le membre 4 peut enregistrer le même objet dans la base de données.

Ainsi, les quatre modules travaillent avec une seule et même structure de données.

---

6. RÈGLES À RESPECTER PAR TOUS LES MEMBRES

1. Ne pas modifier les noms des propriétés définies dans ce document sans accord de l'équipe.
2. Ne pas remplacer un type par un autre sans validation collective.
3. Ne pas créer une deuxième interface pour représenter une structure déjà définie ici.
4. Importer les interfaces communes depuis le dossier `src/shared/types/`.
5. Si une nouvelle propriété devient nécessaire, la proposer à toute l'équipe avant de l'ajouter.
6. Toute modification validée doit être intégrée dans le fichier TypeScript commun et communiquée aux cinq membres.

7. ORGANISATION DU FICHIER COMMUN

Le projet doit contenir :

```
src/
└── shared/
    └── types/
        └── index.ts
```

Ce fichier `index.ts` contiendra les interfaces et les types validés par toute l'équipe.

Tous les membres devront importer leurs types depuis ce fichier.

RÈGLE FINALE

Les membres sont libres d'organiser leur code comme ils le souhaitent, mais les données échangées entre leurs modules doivent respecter exactement les noms et les types définis dans ce document.

Ce document est notre référence commune pour l'intégration de MISSION FORGE.